# COGS Negotiation Agent — How It Works (End to End)

A Databricks-native, agentic application that helps **Kroger category negotiators** prepare for and run supplier COGS (cost-of-goods-sold) negotiations across the beverage category (PepsiCo, Coca-Cola, Keurig Dr Pepper).

It is built entirely on real Databricks primitives — Unity Catalog, Metric Views, Genie, Vector Search, Mosaic AI Model Serving, Lakebase, AI/BI dashboards, and a Databricks App — with a **pluggable LLM layer** (Mosaic AI Gateway ⇔ LiteLLM).

---

## 1. What it does

A negotiator can:

| Capability | What it produces | Powered by |
|---|---|---|
| **Negotiator (AI)** | Ask anything in plain English; a tool-calling agent picks & chains tools (with a visible tool trace) | Hybrid agent (Analytics tool-calling agent + Rehearsal agent) |
| **Supplier Scorecard** | Live metrics table + conversational Q&A + an AI/BI dashboard | Genie + AI/BI |
| **Negotiation Brief** | Board-ready talking points grounded in the supplier's contract | Knowledge Assistant (RAG) |
| **Fact-Pack Deck** | A **fixed 15-slide / 5-act** negotiation deck, every number grounded in a table or contract clause; viewable as a scrolling page or ▶ **Present** (16:9 slides) | Deck builder (manifest + per-act structured output, grounded via `deck_data`) |
| **Rehearsal Room** | Role-play practice against the supplier's account manager | Vendor Rehearsal agent (isolated persona) |

---

## 2. Architecture (three layers)

```
flowchart TB
  subgraph L1["Layer 1 · Experience — Databricks App (React + FastAPI)"]
    UI["Negotiator chat · Deck Hub · Scorecard · Brief · Deck (▶ Present) · Rehearsal"]
  end

  subgraph L2["Layer 2 · Runtime"]
    ROUTER["Persona router (2-way: analytics vs rehearse)"]
    ANALYST["Analytics Agent — tool-calling (bind_tools + executor loop)"]
    REHEARSE["Rehearsal Agent — isolated adversarial persona"]
    TOOLS["Tools: query_scorecard (Genie) · retrieve_contract_clauses (VS) · build_deck"]
    LLM["Pluggable LLM factory  get_llm()"]
    SERVED["Agent-as-model (Mosaic AI Model Serving)"]
  end

  subgraph L3["Layer 3 · Data & Governance — Unity Catalog"]
    DELTA["Delta tables (scorecard, regional, commodity, competitive, macro, sku_waterfal
    GENIE["Genie space (NL→SQL)"]
    DD["deck_data — deterministic warehouse SQL"]
    VS["Vector Search index over contract MSAs"]
    VOL["Volume (contract docs)"]
    LB["Lakebase (Postgres OLTP)"]
    BI["AI/BI dashboard"]
  end

  PROV["Foundation Models: Claude Sonnet 4.5 (FMAPI)  ·  or LiteLLM proxy"]

  UI --> ROUTER
  ROUTER --> ANALYST --> TOOLS
  ROUTER --> REHEARSE --> LLM
  TOOLS --> GENIE --> DELTA
  TOOLS --> VS --> VOL
  ANALYST --> LLM --> PROV
  TOOLS -- build_deck grounds via --> DD --> DELTA
  UI --> GENIE
  UI --> LB
  UI --> BI --> DELTA
  SERVED --> LLM
```

- **Layer 1 — Experience:** the Databricks App users interact with.
- **Layer 2 — Runtime:** a thin **persona router** sends each request to one of two agents — a genuine **tool-calling Analytics Agent** (owns scorecard/brief/deck/chat;
  binds Genie, Vector Search, and the deck builder as tools and chains them) or an **isolated Rehearsal Agent** (adversarial vendor persona, its own prompt/

temperature).
Both call the pluggable LLM factory; the whole thing is also packaged as a served model.

- **Layer 3 — Data & Governance:** everything in Unity Catalog — the single governance boundary. The deck builder grounds on live tables via `deck_data` **deterministic SQL** (Genie is the app's NL→SQL convenience, not a deck dependency).

---

## 3. Repository layout

```
cogs-negotiation-agent/
├── app.py                      # FastAPI entry — mounts API + serves the React SPA; opens
├── app.yaml                    # Databricks App config + all env vars (LLM_PROVIDER, GENI
├── pyproject.toml / uv.lock    # Python deps (app runtime)
├── agent_model.py              # The agent packaged as an MLflow ChatAgent (for serving)
├── deploy_agent.py             # Log → register (UC) → deploy the served model
├── data_foundation.py          # Builds core Delta tables + Metric View
├── build_market_data.py        # Builds the 4 market/cost tables (commodity, competitive,
├── build_genie.py              # Authors the Genie space (all 7 tables)
├── build_knowledge.py          # Builds contract Volume + chunks + Vector Search index
├── build_dashboard.py          # Builds/updates + publishes the AI/BI dashboard
├── eval_agent.py               # Quality Loop (offline): split analytics + rehearsal LLM-
├── monitor_agent.py            # Quality Loop (online): scheduled judges on live traces
├── server/
│   ├── config.py               # Dual/triple-mode auth (local CLI / App SP / model-servin
│   ├── llm.py                  # ★ Pluggable LLM factory — get_llm(), Mosaic ↔ LiteLLM
│   ├── data.py                 # Synthetic source-of-truth data (mirrored into UC)
│   ├── agents.py               # build_negotiation_brief (RAG) · build_fact_pack (15-slid
│   ├── analytics_agent.py      # ★ Genuine tool-calling agent (bind_tools + executor loo
│   ├── rehearsal_agent.py      # Isolated adversarial-vendor persona (temp 0.7)
│   ├── tools.py                # LangChain tools: query_scorecard · retrieve_contract_cla
│   ├── deck_data.py            # Deterministic per-slide warehouse SQL for the deck
│   ├── supervisor.py           # Thin 2-way persona router (rehearse vs analytics)
│   ├── genie.py                # Genie Conversation API client
│   ├── knowledge.py            # Vector Search retrieval (RAG)
│   ├── state.py                # Lakebase (Postgres) persistence + in-memory fallback
│   └── routes/                 # FastAPI routers: catalog, agentic, genie_routes, supervi
└── frontend/                   # React + Vite + TS + Tailwind
    └── src/pages/              # Hub, Negotiator (shows tool trace), Scorecard, Brief, De
```

---

# 4. Component-by-component

## 4.1 The pluggable LLM factory — `server/llm.py` ★

The heart of the design. Every chain/agent calls `get_llm()` and receives a LangChain `BaseChatModel`; **none of them know which provider is live.**

- `LLM_PROVIDER=mosaic` (default) → `ChatOpenAI` pointed at the **governed Databricks path**. On this Azure workspace the dedicated `ai-gateway.*` subdomain isn't available, so it falls back to the workspace `/serving-endpoints` OpenAI-compatible route (usage still records in `system.serving.endpoint_usage`; Gateway config attaches at the endpoint).
- `LLM_PROVIDER=litellm` → the same `ChatOpenAI` pointed at a **LiteLLM proxy** (`LITELLM_BASE_URL`). LiteLLM can route upstream *through* Mosaic to keep one governance boundary, or go direct.
- Both providers are OpenAI wire-compatible, so the switch is **one env var, zero code change**. A third `databricks` mode (native `ChatDatabricks`) exists behind a flag.

Switching is config-only — change `LLM_PROVIDER` in `app.yaml` (app) or the serving endpoint env (served model) and redeploy.

## 4.2 Auth — `server/config.py`

Detects the runtime and authenticates accordingly:
- **Local dev** → Databricks CLI profile (`DATABRICKS_PROFILE`).
- **Databricks App** → auto-injected service-principal credentials.
- **Model serving** → automatic-auth identity (scoped to declared resources).

## 4.3 Data foundation (Layer 3) — `data_foundation.py` → `kroger_demo.cogs`

- **`supplier_scorecard`** (Delta) — one row per supplier with spend, COGS/unit, landed-cost index, YoY changes, trade funds, fill rate, OTIF, rebate tier, contract expiry.
- **`regional_performance`** (Delta) — dollar sales, YoY, units, store counts by US Census division.
- **`landed_cost_metrics`** (**Metric View**) — the *certified* definitions of Total Spend, Avg COGS/Unit, Blended Landed Cost Index, Blended COGS Inflation, etc. Queried with `MEASURE(...)`. This is the single source of truth that Genie and the dashboard agree on.

**Market & cost-breakdown tables** (`build_market_data.py`, seed 8451) — added to ground the deck's Market Context and Cost Breakdown acts in real data:
- **`commodity_input_costs`** — 24-month index series (PET resin, aluminum,

sugar/HFCS, concentrate, corrugate, diesel freight, natural gas), base 100 + YoY.
- `competitive_benchmarks` — shelf-price benchmarks vs competitors + alt-supplier quotes + price gaps, by product and region.
- `macro_indicators` — FX (USD/MXN, USD/EUR), tariffs, labor inflation, supply-chain disruption index, by quarter, each with a negotiation `exposure_note`.
- `sku_cost_waterfall` — per-SKU materials→manufacturing→packaging→freight→duty → landed, plus `prior_year_landed` / `proposed_landed` (drives the YoY bridge, margin, and scenario slides).

> The numbers originate in `server/data.py` / `build_market_data.py` so the app and
> UC stay in sync; in a real deployment these tables come from ingestion + a certified metric layer.

### 4.4 Genie space — `build_genie.py`

A Genie space ( `id 01f1642219cf135cb84f7e0dbc8d6957` ) over the metric view + tables, with instructions encoding the domain rules (use `MEASURE()`, landed-cost-index baseline = 100, positive YoY = cost inflation), example SQL, and benchmark questions. Turns plain-English questions into governed SQL.

### 4.5 Knowledge Assistant (RAG) — `build_knowledge.py` + `server/knowledge.py`

- Per-supplier **Master Supply Agreements** (6 clauses each: Pricing, COGS Adjustment, Rebates, Trade Funds, Service Levels, Term/Termination) written to a UC **Volume** `/Volumes/kroger_demo/cogs/contracts/`.
- Clause-level **chunks** in Delta `contract_chunks` (Change Data Feed on).
- A **Delta-Sync Vector Search index** `contract_chunks_index` with **managed embeddings** ( `databricks-gte-large-en` ) on the existing `kroger-recipe-search` endpoint.
- `knowledge.retrieve()` pulls the top-k clauses *filtered to the supplier*; the Brief agent injects them and **cites the section names**.

### 4.6 Builders & tools — `server/agents.py`, `server/tools.py`, `server/deck_data.py`

The capability functions, all on `get_llm()`:
- `build_negotiation_brief` ( `agents.py` ) — RAG-grounded brief (retrieves contract clauses → prompt → brief that cites clauses + numbers).
- `build_fact_pack` ( `agents.py` ) — the **fixed 15-slide / 5-act** deck builder.

A module-level `DECK_MANIFEST` is the single source of truth for slide number/title/act/grounding. Generation is **per-act** (5 calls) using `get_llm(...).with_structured_output(DeckAct)` over pydantic models (`SlideContent`, `DataCallout`, `ChartSpec`), each injected with that act's manifest slice + its **grounded data** from `deck_data`. A validate/repair pass snaps titles back to the manifest, retries an act once, and fills any still-missing slide with a structured placeholder — so it **never returns a deck missing a slide**. Output: `{title, subtitle, objective, supplier, acts:[{act, slides:[…]}], format_version:"15-slide-v1"}`. *(Gotcha fixed: act generation needs* `max_tokens=8192` *and a downsampled commodity series, or it truncates at the 4096 output cap and an act degrades to placeholders.)*
- `rehearse_turn` (`agents.py`) — thin shim delegating to `rehearsal_agent`.

**Tools (`server/tools.py`)** — the Analytics Agent's bound tools: `query_scorecard(question)` → Genie, `retrieve_contract_clauses(query, supplier_key)` → Vector Search, `build_deck(supplier_key, objective, scorecard_facts, clauses)` → `build_fact_pack`. Structured side-outputs (SQL/rows/parsed deck) are captured via a `contextvars`-backed `ToolContext` so the LLM-facing tool schemas stay clean.

`server/deck_data.py` — deterministic per-slide warehouse SQL (statement execution on warehouse `a455a68035c1f578`; every cell coerced via `_num()` since the API returns strings). Functions: `partnership_overview`, `commodity_trends`, `competitive_landscape`, `macro_factors`, `cost_waterfall`, `yoy_bridge`, `margin_impact`, `trade_funding`. Each degrades to `[]` / `{}` if a table is missing, so the deck builds even before the data load runs.

## 4.7 Hybrid agent — persona router + Analytics Agent + Rehearsal Agent

Instead of one classify-then-dispatch supervisor, the runtime is a **hybrid** split along persona/risk lines (pure LangChain, no LangGraph):

- `server/supervisor.py` — a thin **2-way persona router**: detect rehearsal intent (+ supplier) → `rehearsal_agent`; everything else → `analytics_agent`. Returns the unified envelope the UI renders by `route` (text / table+SQL / deck).
- `server/analytics_agent.py` — a **genuine tool-calling agent**: `get_llm().bind_tools([query_scorecard, retrieve_contract_clauses, build_deck])` plus a manual executor loop (≤4 iterations). The LLM *decides* which tools to call and **chains** them — e.g. its DECK PROTOCOL forces `query_scorecard → retrieve_contract_clauses → build_deck` so a deck is grounded in live numbers +

cited clauses. It returns a `tool_trace` ( `{tool, args}[]` ) the Negotiator UI renders as a "[?] Tools used" strip. Neutral, grounded, pro-Kroger persona.

- `server/rehearsal_agent.py` — the adversarial supplier-KAM persona, **isolated** in its own module with its own system prompt and temperature (0.7) so it can be prompted, guard-railed, and evaluated independently of the analytics surfaces (a single shared prompt can't be both a neutral analyst and a tough vendor).

> Why hybrid, not one tool-calling agent: the rehearsal persona genuinely conflicts with the analyst persona, so it gets its own agent; everything else coheres under one tool-calling agent. Most production "multi-agent" systems split this way — along persona/risk/governance lines, not one-agent-per-feature.

### 4.8 Agent-as-model — `agent_model.py` + `deploy_agent.py`

The same logic packaged as an **MLflow** `ChatAgent`, registered in Unity Catalog ( `kroger_demo.models.cogs_negotiation_agent` ) and deployed to a **Mosaic AI Model Serving** endpoint ( `agents_kroger_demo-models-cogs_negotiation_agent` ). This gives the agent its own queryable REST endpoint, inference tables, evaluation, and monitoring.

- Driven by `custom_inputs.task` = `brief | deck | rehearse | chat` .
- **Routing inside** `predict()` **:** `rehearse` → rehearsal agent; `deck` → `build_fact_pack` **directly,** `brief` → `build_negotiation_brief` **directly** (the deterministic builders, *not* the tool-calling agent); `chat` /NL → the Analytics Agent. This is deliberate: on a Model Serving endpoint the agent's first step ( `query_scorecard` → Genie) fails under the automatic-auth identity, so routing `deck` through the agent would make it bail to prose. The builders are Genie-free — the deck grounds via `deck_data` **direct warehouse SQL** and the brief via Vector Search — so the served deck is fully grounded *and* the path also works for eval/monitoring (which invoke as an SP with no user context).
- Declares the FM endpoint, embedding endpoint, Vector Search index, Genie space, the SQL warehouse, **and the 7** `kroger_demo.cogs` **tables** ( `DatabricksTable` ) as **serving resources**. Declaring the tables grants the automatic-auth identity `SELECT` , so `deck_data` SQL succeeds and the served deck grounds on live data (verified: HTTP 200, 15/15 slides, 0 placeholders, callouts sourced to `commodity_trends` / `cost_waterfall` ).
- **The LLM switch is preserved** here too — set `LLM_PROVIDER` as the endpoint env.
- **OBO deferred:** the served model uses a single automatic-auth identity (no per-user RLS). On-behalf-of-user auth (so a Pepsi buyer ≠ Coke buyer) is the next governance milestone; on Model Serving it's Public Preview + admin-gated and would
also bypass eval/monitoring, so per-user OBO is better realized on the Apps path.

### 4.9 Lakebase persistence — `server/state.py`

Saved briefs/decks/rehearsals + the work queue persist to **Lakebase (Postgres OLTP)** instance `cogs-lakebase`, table `agent_artifacts(id, kind, supplier_key, payload jsonb, created_at)`. Tokens are minted fresh per connection via `w.database.generate_database_credential(...)` (no background refresh). If Lakebase
isn't attached, it falls back to an in-memory store so the app still runs.

### 4.10 AI/BI dashboard — `build_dashboard.py`

A published **Lakeview dashboard** ( `id 01f1643845191b049c9a3acf7531495b` ) over `kroger_demo.cogs`: KPI counters, supplier comparison bars, regional bars, a detail table, and a **global-filters page** (Supplier / Rebate Tier / Region). It's **embedded** in the Scorecard page via iframe (workspace embedding policy set to `ALLOW_APPROVED_DOMAINS` for the app's domain) with an "open full dashboard" link.

### 4.11 Quality Loop — `eval_agent.py` + `monitor_agent.py`

The deployed agent is observed and judged, not just shipped:

- **Inference tables (automatic).** `agents.deploy()` enables request/response + trace logging to `kroger_demo.models.cogs_negotiation_agent_payload`. Every call to the served agent is captured.
- **Offline eval with LLM judges** ( `eval_agent.py` ). An eval dataset (brief / scorecard / chat / rehearse cases with `expected_facts` ) is run through the served endpoint and scored by Mosaic AI / MLflow judges: **Correctness, RelevanceToQuery, Safety**, plus two custom **Guidelines** judges ( `grounded_numbers` , `briefs_cite_contract` ). Logged to an MLflow evaluation run for version-over-version comparison. *(First run: relevance/safety/grounded = 1.0, briefs-cite-contract = 0.8, correctness = 0.75.)*
- **Online monitoring** ( `monitor_agent.py` ). Scheduled scorers (Safety, RelevanceToQuery, grounded_numbers) registered on the agent's MLflow experiment
  so **sampled production traffic is judged continuously**.
- **In-app human feedback.** 👍/👎 + an optional comment on every Brief, Deck, and Negotiator response ( `frontend/src/components/Feedback.tsx` → `POST /api/feedback` ),
  persisted to the Lakebase `feedback` table. The Databricks **Review App** (auto-created at deploy) additionally lets SMEs label responses.
- **Promote feedback → eval.** `promote_feedback.py` turns thumbs-down + corrections
  into new `expected_facts` eval cases, so human signal feeds the offline judge run.

- **Re-register.** When eval regresses, `deploy_agent.py` registers the next model version and redeploys — closing the loop.

> Note: AI Gateway *usage tracking* isn't supported for the agent endpoint type in this workspace; the inference table provides the equivalent observability.

### 4.12 The app — `app.py`, `server/routes/`, `frontend/`

- **Backend** (FastAPI): serves `/api/*` and the built React SPA. Routers: `catalog` (hub/scorecard/overview/health), `agentic` (brief/deck/rehearse + saved artifacts), `genie_routes` (ask/scorecard/dashboard), `supervisor_routes` (the router). The lifespan opens/closes the Lakebase pool.
- **Frontend** (React + Tailwind, dark teal/orange theme): **Deck Hub** (supplier gallery), **Negotiator** (supervisor chat), **Scorecard** (Genie table + Ask + embedded AI/BI dashboard), **Brief** (with clause-citation chips), **Deck**, **Rehearsal**. The sidebar shows a live **LLM-routing badge**.

---

## 5. End-to-end request flows

**A negotiator asks the Negotiator: "draft a brief for Pepsi to claw back COGS"**

```
Browser → /api/supervisor/ask
  → persona router: not rehearse → analytics_agent.run(...)
  → Analytics Agent (bind_tools + loop) picks retrieve_contract_clauses → build_negotiati
      → knowledge.retrieve(...)  → Vector Search index → top contract clauses
      → ChatPromptTemplate | get_llm()  → Claude (Mosaic) → grounded brief
  → returns {route:"analytics", answer, tool_trace} → UI renders brief + "⍰ Tools used"
```

**A negotiator asks: "build a deck for Pepsi" (the 3-hop tool chain)**

```
Browser → /api/supervisor/ask → analytics_agent.run(...)
  → tool 1: query_scorecard(...)            → Genie → live metrics
  → tool 2: retrieve_contract_clauses(pepsi) → Vector Search → citable clauses
  → tool 3: build_deck(pepsi, …, scorecard_facts, clauses)
      → build_fact_pack → 5 per-act with_structured_output calls,
        each grounded by deck_data warehouse SQL → 15-slide deck JSON
  → returns {route:"analytics", deck, tool_trace:[query_scorecard, retrieve_contract_clau
  → UI renders the deck (scroll or ▶ Present) + the tool trace
```

**The Scorecard table loads**

```
Browser → /api/genie/scorecard
  → genie.ask(fixed question)  → Genie Conversation API
      → Genie writes SQL over kroger_demo.cogs → runs on the warehouse
  → returns {text, sql, columns, rows} → UI renders table + "Show SQL"
```

**Someone queries the served model directly**

```
POST /serving-endpoints/agents_…/invocations
  {messages, custom_inputs:{task:"brief", supplier_key:"kdp"}}
  → ChatAgent.predict → build_negotiation_brief → Vector Search + get_llm() → brief
  → inference tables capture the request/response
```

---

# 6. Deploy & run

## Prerequisites

- Databricks CLI authenticated to the workspace (profile `cogs-demo`)
- `uv` (Python) and `node` / `npm` (frontend)
- Serverless workspace (for Lakebase + Apps)

## One-time data/AI build (workspace-side)

For a **fresh workspace**, run `python bootstrap.py` — it executes all five steps below in dependency order, auto-creates the Vector Search endpoint, and captures created IDs (Genie space, dashboard, warehouse) into `deploy_state.json` (see README → "Deploy to a new workspace"). To run steps individually against cogs-demo:

```
DATABRICKS_CONFIG_PROFILE=cogs-demo python data_foundation.py     # core Delta + Metric V
DATABRICKS_CONFIG_PROFILE=cogs-demo python build_market_data.py   # commodity/competitive
DATABRICKS_CONFIG_PROFILE=cogs-demo python build_genie.py         # Genie space (all 7 ta
DATABRICKS_CONFIG_PROFILE=cogs-demo python build_knowledge.py     # Volume + Vector Searc
DATABRICKS_CONFIG_PROFILE=cogs-demo python build_dashboard.py     # AI/BI dashboard
```

### The app

```
# build frontend
cd frontend && npm install && npm run build && cd ..
# lock deps
uv lock --native-tls
# sync + deploy
databricks sync . /Users/<you>/cogs-negotiation-agent --exclude node_modules --exclude .v
databricks apps deploy cogs-negotiation-agent \
  --source-code-path /Workspace/Users/<you>/cogs-negotiation-agent -p cogs-demo
```

Attach resources (UI or CLI): the **LLM serving endpoint** ( `CAN_QUERY` ) and the **Lakebase database** ( `CAN_CONNECT_AND_CREATE` ).

### The served model

```
# needs extra local tooling (not app runtime deps):
uv pip install mlflow databricks-agents azure-storage-file-datalake azure-storage-blob az
DATABRICKS_CONFIG_PROFILE=cogs-demo python deploy_agent.py
```

### Local dev

```
export DATABRICKS_PROFILE=cogs-demo
uv run uvicorn app:app --reload --port 8000      # backend
cd frontend && npm run dev                        # frontend (proxies /api → :8000)
```

---

## 7. Switching the LLM provider (Mosaic ↔ LiteLLM)

In `app.yaml` (app) or the serving endpoint env (served model):

```
# Mosaic (default, governed)
LLM_PROVIDER: mosaic
LLM_MODEL:    databricks-claude-sonnet-4-5

# LiteLLM
LLM_PROVIDER: litellm
LITELLM_BASE_URL: http://<litellm-proxy>:4000
LITELLM_API_KEY:  {{secrets/<scope>/<key>}}
LITELLM_MODEL:    claude-sonnet-4
LITELLM_ROUTES_VIA_MOSAIC: "true"   # keep one governance boundary
```

Then redeploy. No agent code changes.

---

## 8. Design decisions & known limitations

- **Synthetic-but-real data.** Numbers originate in `server/data.py` / `build_market_data.py` and are materialized as real Delta tables; swap ingestion + a certified metric layer for production.
- **Hybrid agent, not one-agent-per-feature.** A 2-way persona router splits the coherent analyst surfaces (one tool-calling agent) from the conflicting adversarial rehearsal persona (its own agent). Tool selection is the LLM's job, not hardcoded.
- **Deck is code-owned structure, LLM-filled content.** A fixed 15-slide manifest + per-act structured output + validate/repair guarantees the format; every numeric callout is sourced to a table or contract clause (no fabrication).
- **Served model is Genie-free** — `deck` / `brief` run the deterministic builders (deck grounds via `deck_data` direct SQL, declared `DatabricksTable` resources give
the auto-auth identity `SELECT`); the app's Analytics Agent uses Genie for NL→SQL.
- **OBO deferred.** The served model is single-identity (no per-user RLS); OBO is the next governance step, best done on the Apps path.
- **Regional viz uses bars, not a choropleth** — the data is by Census division, not US states/countries (a choropleth needs standard region keys).
- **"Avg COGS Inflation"** on the dashboard is a simple average (filter-friendly); the spend-weighted "blended" figure lives in the metric view for exact reporting.
- **Embedding** is enabled with a least-privilege workspace policy ( `ALLOW_APPROVED_DOMAINS` , only the app's domain).

## 9. Resource inventory

| Resource | Identifier |
|---|---|
| Workspace | `adb-7405614449041750.10.azuredatabricks.net` (Azure) · CLI profile `cogs-demo` |
| App URL | `https://cogs-negotiation-agent-7405614449041750.10.azure.databricksapps.com` |
| App service principal | `a432c266-0858-4f97-aa29-4c726a30c0eb` |
| Catalog / schema | `kroger_demo.cogs` |
| Delta tables (core) | `supplier_scorecard` , `regional_performance` , `contract_chunks` |

| Resource | Identifier |
| --- | --- |
| Delta tables (market/cost) | `commodity_input_costs`, `competitive_benchmarks`, `macro_indicators`, `sku_cost_waterfall` |
| Metric View | `kroger_demo.cogs.landed_cost_metrics` |
| Volume | `/Volumes/kroger_demo/cogs/contracts/` |
| Genie space | `01f1642219cf135cb84f7e0dbc8d6957` |
| Vector Search index | `kroger_demo.cogs.contract_chunks_index` (endpoint `kroger-recipe-search`, embeddings `databricks-gte-large-en`) |
| Registered model | `kroger_demo.models.cogs_negotiation_agent` |
| Agent serving endpoint | `agents_kroger_demo-models-cogs_negotiation_agent` |
| Foundation model | `databricks-claude-sonnet-4-5` |
| Lakebase instance | `cogs-lakebase` (`ep-blue-pine-e121z0sk.database.eastus2.azuredatabricks.net`) |
| AI/BI dashboard | `01f1643845191b049c9a3acf7531495b` |
| SQL warehouse | `a455a68035c1f578` (Serverless Starter) |